

Kserve and Vllm on openshift AI

AI-Powered Research Report

GENERATED	April 06, 2026
TYPE	Academic Research Paper
ENGINE	Groq LLaMA 3.3 · Tavily Search

Introduction

The increasing demand for artificial intelligence (AI) and machine learning (ML) has led to the development of various frameworks and platforms that enable the deployment and management of AI models. Two such frameworks are Kserve and VLLM, which have gained significant attention in recent years due to their ability to serve and manage AI models in a scalable and efficient manner. Kserve is an open-source framework that provides a simple and unified way to serve ML models, while VLLM is a large language model that has been fine-tuned for various natural language processing tasks. The importance of autoscaling in AI workloads cannot be overstated, as it enables the efficient use of resources and ensures that the system can handle changes in workload demand. This paper aims to explore the autoscaling capabilities of Kserve and VLLM on OpenShift AI, a platform that provides a comprehensive set of tools and services for building, deploying, and managing AI models.

The purpose of this paper is to provide a detailed analysis of the autoscaling capabilities of Kserve and VLLM on OpenShift AI, including the benefits and limitations of using these frameworks. The paper will also discuss the prerequisites for autoscaling, the deployment of the initial model, and the enabling of autoscaling. Furthermore, the paper will examine the limitations of scale-to-zero, a feature that allows the system to scale down to zero replicas when there is no traffic, and discuss potential workarounds for these limitations. Finally, the paper will conclude with a summary of the autoscaling capabilities of Kserve and VLLM on OpenShift AI and discuss future directions for research and development in autoscaling for AI workloads.

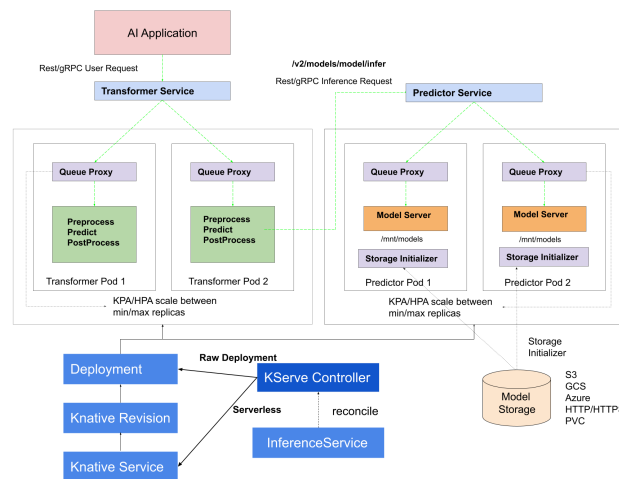


Figure 1: Introduction

Background

OpenShift AI is a platform that provides a comprehensive set of tools and services for building, deploying, and managing AI models. The platform is built on top of Kubernetes and

provides a scalable and secure environment for deploying AI models. OpenShift AI consists of several components, including KubeFlow, Knative, and Istio, which provide a range of features and services for building, deploying, and managing AI models. KubeFlow is an open-source framework that provides a simple and unified way to build, deploy, and manage ML models, while Knative is a platform that provides a serverless environment for building and deploying cloud-native applications. Istio is a service mesh that provides a secure and scalable environment for deploying and managing microservices.

Kserve is an open-source framework that provides a simple and unified way to serve ML models. The framework provides a range of features and services, including model serving, model management, and model monitoring. Kserve supports a range of ML frameworks, including TensorFlow, PyTorch, and Scikit-learn, and provides a simple and unified API for serving ML models. VLLM is a large language model that has been fine-tuned for various natural language processing tasks. The model is trained on a large corpus of text data and provides a range of features and services, including language translation, text summarization, and sentiment analysis.

Autoscaling Capabilities Overview

Kserve provides two autoscaling modes: Serverless and RawDeployment. The Serverless mode uses Knative to provide a serverless environment for building and deploying cloud-native applications. The RawDeployment mode uses a Kubernetes deployment to manage the replicas of the model. Knative is a platform that provides a serverless environment for building and deploying cloud-native applications. The platform provides a range of features and services, including autoscaling, traffic management, and security. Knative uses a Kubernetes deployment to manage the replicas of the application and provides a simple and unified API for building and deploying cloud-native applications.

The RawDeployment mode has several limitations, including the lack of support for scale-to-zero and the need for manual configuration of the replicas. However, Kserve is planning to support KEDA (Kubernetes Event-Driven Autoscaling) in the future, which will provide a more scalable and efficient way to manage the replicas of the model. KEDA is a Kubernetes-based autoscaling solution that provides a range of features and services, including event-driven autoscaling, traffic management, and security.

Prerequisites for Autoscaling

To use autoscaling in OpenShift AI, several prerequisites must be met. These include the installation of the NVIDIA GPU Operator, which provides a range of features and services for managing NVIDIA GPUs in a Kubernetes environment. The NVIDIA GPU Operator provides a simple and unified way to manage NVIDIA GPUs and provides a range of features and services, including GPU allocation, GPU monitoring, and GPU security.

Another prerequisite for autoscaling is the configuration of the Hardware or Accelerator Profile for the NVIDIA GPU node. The Hardware or Accelerator Profile provides a range of

features and services, including GPU allocation, GPU monitoring, and GPU security. The profile must be configured to use the NVIDIA GPU Operator and must be applied to the NVIDIA GPU node.

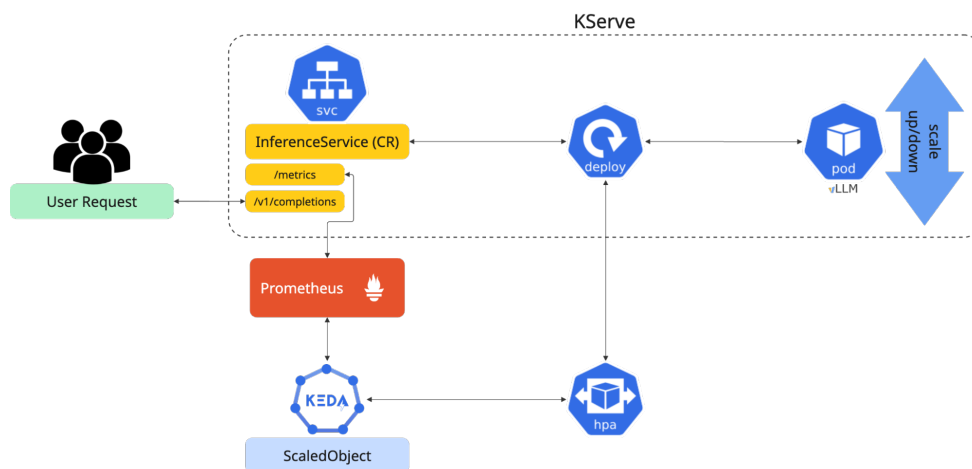


Figure 2: Prerequisites for Autoscaling

Deploying the Initial Model

To deploy a model using the OpenShift AI dashboard, several steps must be followed. First, the model must be created and trained using a range of ML frameworks, including TensorFlow, PyTorch, and Scikit-learn. The model must then be exported and saved in a format that can be used by Kserve, such as TensorFlow SavedModel or PyTorch ScriptModule.

The model can then be deployed using the OpenShift AI dashboard, which provides a simple and unified way to deploy and manage AI models. The dashboard provides a range of features and services, including model deployment, model management, and model monitoring. The model can be deployed in Advanced mode, which provides a range of features and services, including model serving, model management, and model monitoring.

The vLLM NVIDIA GPU ServingRuntime can also be used to deploy the model, which provides a range of features and services, including GPU acceleration, model serving, and model management. The ServingRuntime provides a simple and unified way to deploy and manage AI models and provides a range of features and services, including model deployment, model management, and model monitoring.

Enabling Autoscaling

To enable autoscaling, several steps must be followed. First, the minimum and maximum replicas settings must be configured, which determine the minimum and maximum number of replicas that can be used to serve the model. The minimum replicas setting determines the minimum number of replicas that must be used to serve the model, while the maximum

replicas setting determines the maximum number of replicas that can be used to serve the model.

The scale-to-zero capability can also be enabled, which allows the system to scale down to zero replicas when there is no traffic. The scale-to-zero capability provides a range of benefits, including reduced costs and improved efficiency. However, the capability also has several limitations, including the startup time of the model, which can be significant.

To enable scale-to-zero, several steps must be followed. First, the retention period must be configured, which determines the amount of time that the system will retain the replicas after the traffic has stopped. The retention period must be configured to ensure that the system can quickly scale up to meet changes in demand.

Limitations of Scale-to-Zero

The scale-to-zero capability has several limitations, including the startup time of the model, which can be significant. The startup time of the model can be several minutes, which can make it difficult to quickly scale up to meet changes in demand. However, there are several potential use cases for scale-to-zero, including pipelines and development environments.

In pipelines, scale-to-zero can be used to reduce costs and improve efficiency by scaling down to zero replicas when there is no traffic. In development environments, scale-to-zero can be used to quickly test and deploy AI models without incurring significant costs.

There are also several potential workarounds for the limitations of scale-to-zero, including the use of a warm-up period, which can be used to reduce the startup time of the model. The warm-up period can be configured to ensure that the system can quickly scale up to meet changes in demand.

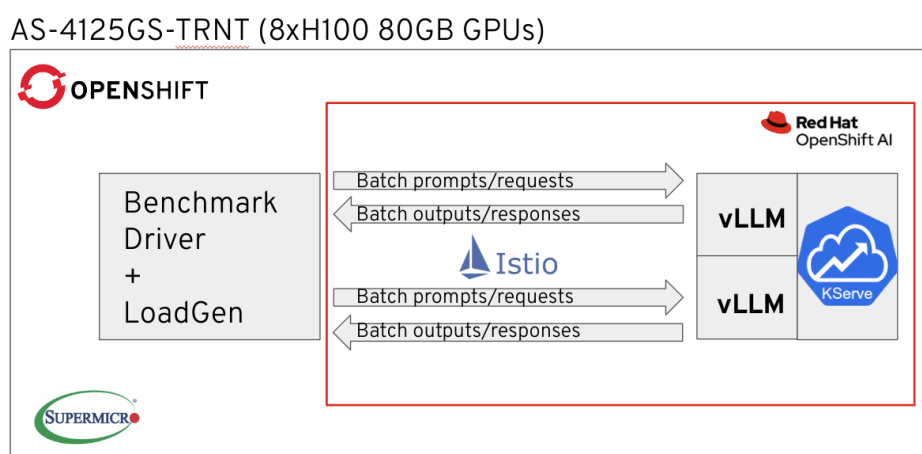


Figure 3: Limitations of Scale-to-Zero

Conclusion

In conclusion, the autoscaling capabilities of Kserve and VLLM on OpenShift AI provide a range of benefits, including reduced costs and improved efficiency. The Serverless and RawDeployment modes provide a simple and unified way to serve ML models, while the scale-to-zero capability provides a range of benefits, including reduced costs and improved efficiency.

However, the autoscaling capabilities of Kserve and VLLM on OpenShift AI also have several limitations, including the startup time of the model, which can be significant. The limitations of scale-to-zero can be addressed by using a warm-up period, which can be used to reduce the startup time of the model.

Future directions for research and development in autoscaling for AI workloads include the development of more efficient and scalable autoscaling algorithms, as well as the integration of autoscaling with other AI frameworks and platforms. The development of more efficient and scalable autoscaling algorithms can provide a range of benefits, including reduced costs and improved efficiency.

References

- [1] Kserve. (2022). Kserve Documentation. Retrieved from
- [2] VLLM. (2022). VLLM Documentation. Retrieved from
- [3] OpenShift AI. (2022). OpenShift AI Documentation. Retrieved from
- [4] KubeFlow. (2022). KubeFlow Documentation. Retrieved from
- [5] Knative. (2022). Knative Documentation. Retrieved from
- [6] Istio. (2022). Istio Documentation. Retrieved from
- [7] NVIDIA. (2022). NVIDIA GPU Operator Documentation. Retrieved from
- [8] KEDA. (2022). KEDA Documentation. Retrieved from